

Names: Rhilo Sotto, John Hayali, Eren Ugur

Report for Programming Assignment #1

Explanation of Implementation of Different Cases

Baseline

The baseline implementation does the summation sequentially by using a single loop from 0(N_start) to N-1(N_end). Timing was measured with `clock_gettime(CLOCK_MONOTONIC)`. This was the shortest and simplest code having just one loop, timing calls, and a print statement.

Multithreading

The multithreading implementation uses the pthread library. We divided the range of numbers among the various threads and assigned each thread a portion of the numbers and calculated a partial sum. After the threads finish, the main program collects the partial results and adds them together to get the final answer. This method was more efficient and still not hard to implement since all threads shared the same memory

Multitasking

For multitasking we implemented two versions which are using `fork()` to create child processes and using `popen()` to start new processes. For the `fork()` version, each process calculated part of the sum and then sent the results back to the parent process using pipes. The parent process then combined all the results together. This worked but required more code because processes do not share memory and need pipes for communication. In the `popen()` version, the parent program read each result from the process's output and added them together. The code for this was easier to write than the first version but it had more overhead since it used standard input/output instead of memory sharing

Code Sizes of Multithreading and Multitasking

The multitasking version was longer because of the extra code needed for pipes and process management for the `fork()` while the `popen()` version was a little shorter. The multithreading version was shorter than both versions of multitasking because of thread creation and joining

Results

[\[Link to Spreadsheet\]](#)

Execution Time Statistics

For each implementation, we performed 100 tests. Specifically, we did...

- 100 tests for the baseline
- 300 tests for multithreading with 2, 4, and 8 threads
- 300 tests for multiprocessing via fork() with 2, 4, and 8 processes
- 300 tests for multiprocessing via popen() with 2, 4, and 8 processes

Thus, we performed a total of 1000 tests for each of the three specified values of N (100000000, 1000000000, and 10000000000). In addition, the specifications of the CPU that was a part of the system that ran the tests are “Intel(R) Core(TM) i7-10510U CPU @ 1.80GHz” (4 Cores and 8 Threads). Here is table that displays all of the **averages** for the execution times:

	N = 100000000	N = 1000000000	N = 10000000000
Baseline	0.2603 ns	2.7230 ns	27.5318 ns
Multithreading (2 threads)	0.1217 ns	1.2742 ns	13.3060 ns
Multithreading (4 threads)	0.0955 ns	0.9369 ns	9.6159 ns
Multithreading (8 threads)	0.0750 ns	0.6985 ns	6.6488 ns
Multiprocessing via fork() (2 processes)	0.1272 ns	1.3061 ns	13.5286 ns
Multiprocessing via fork() (4 processes)	0.0921 ns	0.9453 ns	9.5225 ns
Multiprocessing via fork() (8 processes)	0.0823 ns	0.7627 ns	6.2292 ns
Multiprocessing via popen() (2 processes)	0.1439 ns	1.4700 ns	15.4751 ns
Multiprocessing via popen() (4 processes)	0.0836 ns	0.8693 ns	8.8912 ns
Multiprocessing via popen() (8 processes)	0.0723 ns	0.6457 ns	5.5434 ns

Moreover, here is table that displays all of the **standard deviations** for the execution times:

	N = 100000000	N = 1000000000	N = 10000000000
Baseline	0.0036 ns	0.0326 ns	0.4632 ns
Multithreading (2 threads)	0.0027 ns	0.0226 ns	0.0764 ns
Multithreading (4 threads)	0.0035 ns	0.0203 ns	0.1757 ns
Multithreading (8 threads)	0.0037 ns	0.0250 ns	0.1071 ns
Multiprocessing via fork() (2 processes)	0.0030 ns	0.0175 ns	0.1282 ns
Multiprocessing via fork() (4 processes)	0.0028 ns	0.0555 ns	0.4680 ns
Multiprocessing via fork() (8 processes)	0.0107 ns	0.0747 ns	0.1993 ns
Multiprocessing via popen() (2 processes)	0.0056 ns	0.0222 ns	0.2595 ns
Multiprocessing via popen() (4 processes)	0.0035 ns	0.0356 ns	0.2762 ns
Multiprocessing via popen() (8 processes)	0.0039 ns	0.0288 ns	0.1059 ns

By looking at the table, it appears that multiprocessing resulted in the highest standard deviations, while multithreading resulted in the lowest standard deviations. It is possible that multiprocessing experienced higher standard deviations or fluctuations in execution time because it is affected by circumstances such as OS scheduling and extra overhead; multithreading is less influenced by these conditions. These circumstances can make multiprocessing more unpredictable due to the various ways it can be interrupted or slowed down.

Observations of increasing N

As N is increased, the time it takes for the program to execute increases as well, regardless of the implementation. This is because the program has to perform more summation operations as N increases.

Observations of increasing NUM_THREADS

As NUM_THREADS increases, it appears that the time it takes for the program to execute decreases until a point. If NUM_THREADS becomes too large, the time actually increases again and levels out. This is due to the fact that the system's CPU has a limited amount of threads (e.g., the CPU that was used in the experiment has 8 threads), and this amount influences the time it takes for a multitasking program to execute.

Observations of increasing NUM_TASKS

Similar to NUM_THREADS, as NUM_TASKS increases, it appears that the time it takes for the program to execute decreases until a point. If NUM_TASKS becomes too large, the time increases again and levels out. This is due to the fact that the system's CPU has a limited amount of cores (e.g., the CPU that was used in the experiment has 4 cores), and this amount influences the time it takes for a multithreading program to execute.

Multithreading vs. Multitasking

In terms of execution times, it does not seem that there is a clear winner because their times were often similar throughout the experiments. However, in terms of meeting deadlines, the answer to this question would change if the system had less resources and a CPU with less cores (e.g., an RTOS running on a single-core or few-core CPU). In this case, multitasking could be superior to multithreading because if a task runs into an error, it will most likely not affect other running tasks. On the other hand, a thread that runs into an error can cause the entire process to crash. This is particularly important for RTOSs because they aim to achieve a deterministic behavior.

How the implementation would change if NUM_THREADS and NUM_TASKS did not evenly divide N

If NUM_THREADS and NUM_TASKS did not evenly divide N, then that would mean certain threads or tasks would have to perform an extra summation operation. For example, if NUM_THREADS is 4 and N is 99, then three threads would perform 25 summations and the fourth thread would perform 24 operations. To achieve this, the current implementations would have to change such that the total range $[0, N]$ is split as evenly as possible to all. This means that instead of assuming $N/\text{NUM_THREADS}$ or $N/\text{NUM_TASKS}$ are integers, the code would calculate the start and end of each thread or process, making sure that the remainder values are distributed. This is done by adding one extra part to the first few threads or processes until the remainder is covered. By doing this it ensures that no numbers are skipped.